

A SOFTWARE ARTIFACT FOR DETERMINISTIC CONVERSION OF OPENTELEMETRY TRACES INTO STRUCTURED EXECUTION EVIDENCE

ABSTRACT

This software artifact implements a deterministic transformation from OpenTelemetry trace data into structured execution evidence representations. The repository contains the schema, conversion script, validation path, trace provenance record, baseline comparison, and reproducibility script. The evaluation uses synthetic traces, a public OpenTelemetry trace fixture, and a noisy-trace recovery case to check local conversion and validator behavior. The artifact is limited to local trace transformation and schema-bound validation; it does not claim a new telemetry standard, deployment proof, certification, or legal non-repudiation.

CONTRIBUTION STATEMENT (STRICT SCOPE)

This work contributes:

1. A deterministic transformation function:
OTLP trace -> EEOAP evidence object.
2. A schema-bound validation layer:
EEOAP v0.1 JSON schema plus validator-readable evidence output.
3. A reproducible trace grounding pipeline:
OpenTelemetry source fixture -> extraction -> mapping -> evidence generation.

Non-contributions:

- no new observability standard;
- no distributed tracing protocol modification;
- no runtime instrumentation changes;
- no production deployment claims.

ARTIFACT DESCRIPTION

The repository contains a single research artifact:

'EEOAP-Trace-Adapter (v0.1)'.

It includes:

- schema definition;
- trace-to-evidence conversion script;
- validation pipeline;
- reproducibility scripts;
- example OTLP trace input;
- generated EEOAP outputs.

No external services, runtimes, or deployments are required.

PRIMARY ARTIFACT ENTRY POINT

The repository has a single primary entry point:

`'scripts/reproduce_paper.sh'`

Executing this script reconstructs:

1. schema validation;
2. trace conversion;
3. baseline comparison;
4. evaluation tables.

No additional project-specific configuration is required beyond the local repository environment.

EXECUTION ENVIRONMENT

The bundled experiments require a standard Python environment with `'python3'` and `'pytest'`. The package contains the local `'agent_evidence'` source tree and can invoke the CLI through the unpacked source directory when an installed `'agent-evidence'` executable is not available.

No network access is required during reproduction when the declared Python dependencies are already installed in the local environment.

ENVIRONMENT STABILITY BOUNDARY

The bundled results are validated under the local dependency boundary described in `'docs/softwarex_environment.md'`.

The reproducibility claim is bounded to:

- the repository snapshot identified by the package manifest;
- the Python and package metadata recorded for the local validation run;
- local execution without network access;
- semantic equivalence of generated evidence content and validator outcomes.

This boundary is narrower than a claim of universal byte-identical behavior across arbitrary operating systems, Python builds, locale settings, or filesystem layouts.

READINESS STATEMENT

The artifact is executable through a single entry script and does not require

additional configuration beyond a standard Python environment.

All results reported in the manuscript can be reproduced directly from the provided repository snapshot.

SOFTWARE ARTIFACT SCOPE

The contribution is strictly software-centric:

- a Python-based trace transformation code path;
- a reproducible execution pipeline;
- a schema-validated evidence generator.

It is not a protocol proposal, theoretical framework, or standards specification.

WHY THIS MATTERS

Modern agent systems can produce large volumes of OpenTelemetry traces, but raw spans alone do not provide structured post-execution representations that can be checked by a local validator.

This work provides a minimal deterministic mapping between execution telemetry and structured evidence representation, enabling consistent post-run analysis within the repository's local validation boundary.

DISTINCTION FROM DATA PROCESSING PIPELINES

Unlike general data transformation pipelines, this system enforces:

- schema-constrained output validity;
- deterministic mapping rules;
- trace provenance preservation;
- validation-driven execution evidence structure.

PRACTICAL UTILITY

The system is intended for debugging and post-execution analysis of agent systems that generate OpenTelemetry traces.

It provides a structured view of execution behavior that is not available in raw trace formats, while preserving the original trace as the input artifact.

1. MOTIVATION AND INTENDED USERS

OpenTelemetry traces are useful for observability, but a trace alone does not define the operation-accountability evidence expected by this repository's local validator. The adapter is intended for researchers, tool builders, and

reviewers who need to inspect how a telemetry trace can be bound to a reviewable operation statement with explicit input, output, provenance, policy, evidence, and validation fields.

The positive use case is not "another trace format." The positive use case is local post-run review: a reviewer can check whether a trace-derived statement names the operation, binds the raw trace digest, preserves span identifiers, and passes the local schema, reference, consistency, and integrity checks.

CONCRETE USAGE SCENARIO

In a typical agent runtime debugging scenario:

1. A distributed agent system emits OTLP traces during execution.
2. These traces are collected from an OpenTelemetry collector or exported as OTLP JSON.
3. The adapter converts traces into EEOAP evidence objects.
4. Developers inspect failures using structured evidence instead of raw spans.

This enables post-execution failure analysis and review of agent behavior within the repository's local validation boundary.

MINIMAL DEBUGGING WALKTHROUGH

If a developer receives an OTLP JSON export for a failed run, the adapter produces a structured record that surfaces the trace identifier, preserved span identifiers, input digest, output statement digest, validation status, and linked evidence references in one place.

This lets the reviewer answer a concrete question: Which trace produced this reviewable statement, and did the generated evidence pass schema and integrity checks? The reviewer can answer this without manually traversing raw OTLP JSON.

DEBUGGING OUTCOME DEMONSTRATION

The current walkthrough demonstrates local inspection mechanics rather than a field study or incident-response benchmark.

The demonstrated inspection path is:

1. start with the original OTLP JSON fixture;
2. convert it into an EEOAP statement;
3. inspect the preserved trace and span identifiers;
4. inspect the input trace digest and output statement digest;
5. inspect the validator-readable status.

This reduces manual trace traversal for the bounded question of trace identity, provenance binding, and validation status. It does not claim measured time

savings or production incident coverage.

EXAMPLE CLI INVOCATION

```
""bash
python scripts/convert_otel_trace_to_eeoap.py \
  --input data/otel/raw_demo_trace.json \
  --output data/eeoap/real_trace_evidence.json \
  --adapter-record data/eeoap/real_trace_adapter_record.json
""
```

2. EEOAP V0.1 DEFINITION

EEOAP means Execution Evidence and Operation Accountability Profile. In this repository, EEOAP is a local, profile-aware validation surface for completed operations. The canonical statement shape is implemented by:

- 'schema/execution-evidence-operation-accountability-profile-v0.1.schema.json'
- 'spec/execution-evidence-operation-accountability-profile-v0.1.md'
- 'agent-evidence validate-profile'

For this trace-grounded package, the additional record schema is:

- 'specs/eeoap/v0.1/eeoap.schema.json'
- 'docs/specs/EEOAP_v0_1.md'

The trace-grounding schema includes 'execution_id', 'timestamp', 'trace_source', 'spans[]', 'evidence[]', 'status', and 'version = "v0.1"'. This schema is repository-local. It is not an official FDO standard or a standards-body artifact.

WHY A NAMED REPOSITORY-SCOPED SCHEMA

EEOAP is used here as a short name for the repository-scoped structured JSON evidence contract required by the validator and experiments.

The name does not imply external standard status. It labels a stable local field set so that the transformation target, validation surface, and examples can be referenced unambiguously throughout the artifact.

NECESSITY OF THE EEOAP ABSTRACTION

EEOAP is introduced to provide a stable, schema-bound representation of execution evidence derived from OpenTelemetry traces.

Without such a structured intermediate representation, trace data remains limited to observability-oriented inspection and cannot be directly validated against execution-level constraints in this artifact.

The abstraction therefore serves as a minimal contract layer between telemetry data and structured validation outputs.

TASK-BASED NECESSITY CLARIFICATION

The necessity of EEOAP in this work is task-bound rather than ontological.

The target task is not generic trace storage. The target task is post-execution inspection requiring one object that binds:

- trace provenance;
- policy reference;
- evidence references;
- validation outcome;
- execution-level statement fields.

Raw OTLP retention and flat JSON summaries preserve observability data, but they do not by themselves provide this combined review surface in validator-readable form.

3. ARCHITECTURE

The architecture is shown in 'docs/figures/architecture.svg' and represented by the Mermaid source 'docs/figures/architecture.mmd':

```
““text
OpenTelemetry Collector / OTLP JSON Source
-> Trace Parser
-> EEOAP Mapper
-> Evidence Store
-> Audit Interface
```

The EEOAP Mapper also emits a local validator input. The validator result feeds the audit interface.

```
““
```

This design keeps the adapter decoupled from production execution systems. It does not add a hosted API, remote MCP service, policy router, or decision system.

4. REAL OPENTELEMETRY TRACE GROUNDING

The raw input fixture is:

- 'data/otel/raw_demo_trace.json'
- provenance sidecar: 'data/otel/raw_demo_trace.provenance.json'

The fixture is fetched from the public OpenTelemetry protocol repository:

- 'open-telemetry/opentelemetry-proto/examples/trace.json'

The converted EEOAP statement is:

- 'data/eeoap/real_trace_evidence.json'

The trace-grounded adapter record is:

- 'data/eeoap/real_trace_adapter_record.json'

The local validator result for 'data/eeoap/real_trace_evidence.json' is 'ok=true' with 'issue_count=0'.

SUPPORTED OPENTELEMETRY SURFACE

The bundled artifact demonstrates mapping of trace identifiers, parent-child relationships, timestamps, span kind, selected span attributes, resource attributes, and instrumentation scope metadata from OTLP JSON.

The current submission does not claim exhaustive preservation of every OpenTelemetry trace feature. Fields such as events, links, detailed status semantics, and schema URL metadata remain extension points beyond the bundled examples.

OTLP FIELD COVERAGE MATRIX

The supported OTLP JSON surface is defined explicitly at field level.

OTLP field	Current handling	EEOAP role
---	---	---
'traceId'	preserved	trace identity and statement identifier
'spanId'	preserved	span identity in normalized evidence
'parentSpanId'	preserved when present	parent-child reconstruction
'startTimeUnixNano'	mapped	evidence timestamp source
'endTimeUnixNano'	mapped	span timing metadata
'kind'	mapped	span role metadata
span attributes	selected attributes normalized	local evidence context
resource attributes	normalized	source/runtime context
instrumentation scope name/version	normalized	instrumentation provenance context
events	not interpreted	current limitation
links	not interpreted	current limitation
detailed status semantics	not fully interpreted	current limitation
schema URL metadata	retained as an extension point	current limitation

Unsupported fields are not silently claimed as preserved semantics. Their absence should be interpreted as a scoped limitation of the current artifact.

5. INPUT-OUTPUT EXAMPLE

The input-output example is:

- 'examples/io_trace_to_eeoap.json'

It shows one OpenTelemetry span chain, the mapping from trace ID, span ID, timestamp, and raw trace digest into the EEOAP statement, and the resulting validator-readable output fields.

6. EVALUATION

The local experiment suite contains:

- 'experiments/exp1_synthetic.py'
- 'experiments/exp2_real_trace.py'
- 'experiments/exp3_noisy_recovery.py'
- 'experiments/exp4_determinism_oracle.py'

The generated evaluation table is:

- 'paper/figures/evaluation_table.md'

Summary:

Experiment	Source	Span count	Validator ok	Issue count
exp1_synthetic	synthetic fixture	2	true	0
exp2_real_trace	public OpenTelemetry proto example	1	true	0
exp3_noisy_recovery	public example plus trace-format noise	1	true	0
exp4_determinism_oracle	repeated conversion oracle	1	true	0

These results test local conversion and validation behavior only. They do not prove production robustness, benchmark superiority, or external adoption.

The public OpenTelemetry example bundled upstream is intentionally small and contains a single demonstrated span. In this submission it serves as a provenance-tracked real fixture, while the synthetic experiment exercises parent-child preservation and the noisy experiment tests resilience to non-semantic additions.

A broader real-trace suite remains future work and is not claimed here.

REMAINING REAL-TRACE ROBUSTNESS GAP

The package does not yet include an additional public multi-span real trace case beyond the official OpenTelemetry example fixture.

The current multi-span evidence comes from the synthetic experiment, which is useful for parent-child preservation but does not substitute for a larger public real-world telemetry workload.

Before external upload, an additional publicly redistributable multi-span trace would further reduce evaluation-scope risk.

EVALUATION INTERPRETATION

The evaluation is not a performance benchmark. It is a structural correctness evaluation of transformation consistency.

The goal is to verify:

- whether the same trace input produces deterministic EEOAP outputs;
- whether malformed profile or trace-derived structures are rejected by schema-bound validation;
- whether baseline representations lose structural information that is present in the EEOAP output.

VALIDATION INDEPENDENCE CLARIFICATION

Schema validity in this artifact is a structural acceptance criterion, not the sole definition of correctness.

Correctness is assessed through a combination of:

- faithful field transfer from OTLP input;
- provenance retention;
- deterministic content generation;
- schema-bound structural validity.

The validator therefore acts as one bounded check within the evaluation, rather than as a self-sufficient proof of conceptual correctness.

EVALUATION SCOPE JUSTIFICATION

The evaluation is designed to verify structural correctness of the transformation rather than benchmark-scale performance.

The focus is on deterministic mapping fidelity, schema validity, and trace provenance preservation across representative trace conditions.

DETERMINISM BOUNDARY

In this paper, determinism means that a fixed OTLP JSON input processed within a fixed package layout yields the same trace-derived evidence content and the

same validator outcome.

The artifact also records file locators as provenance metadata. These locator strings should not be read as a claim that arbitrary filesystem locations produce byte-identical outputs across all execution environments.

DETERMINISM TEST ORACLE

In this artifact, determinism is assessed by a content-equivalence oracle rather than by byte equality of every generated file.

The oracle compares:

- canonicalized evidence content;
- validator outcome status.

The oracle excludes:

- environment-specific file locator strings.

This distinction prevents provenance bookkeeping fields from being misread as semantic non-determinism.

7. COMPARISON BASELINE

The baseline is raw OpenTelemetry trace retention without an operation accountability statement. That baseline preserves span data but does not, by itself, bind a local policy reference, evidence references, provenance linkage, or a validator-readable EEOAP validation result. The adapter adds those local review surfaces while preserving the original trace as an input artifact.

The comparison is a feature-coverage comparison rather than a scored rubric. The compared surfaces are trace identity preservation, explicit policy binding, provenance linkage, evidence references, and validator-readable output.

BASELINE SCOPE EXPANSION

The comparison is not limited to raw OTLP retention as a straw-man baseline.

The manuscript distinguishes among:

- raw OTLP retention;
- OTLP exported as generic structured JSON;
- EEOAP generation as a validator-readable review artifact.

This separation makes explicit which gains come from normalization alone and which gains come from the added evidence and validation surface.

8. LIMITATIONS

The present evaluation is bounded to repository-local validation and a small bundled trace suite.

Current limitations are:

- one public OTLP JSON fixture;
- no additional public multi-span real trace fixture in the package;
- no container image or lockfile beyond the documented local environment boundary;
- no claim of production robustness or external certification.

9. AVAILABILITY

This repository snapshot contains the manuscript, software components, and support material used for local validation and review preparation.

The software distribution and support material are publicly available at:
<https://github.com/joy7758/agent-evidence/releases/tag/v2.0-softwarex-clean>

DATA AVAILABILITY

The manuscript data statement cites the public repository release record:
<https://github.com/joy7758/agent-evidence/releases/tag/v2.0-softwarex-clean>

The public record includes:

- the software snapshot used for this manuscript;
- the bundled OTLP input fixtures;
- the generated EEOAP outputs;
- the reproducibility script;
- the package manifest with checksums.

If any support material cannot be redistributed, the submission should state the reason explicitly.

KEYWORDS

OpenTelemetry; tracing; evidence; validation; reproducibility

10. CONCLUSION

The artifact provides a bounded, reproducible software path from OpenTelemetry trace input to structured execution evidence and local validation output. Its contribution is the explicit transformation boundary, the validator-readable evidence representation, and the reproducible package used to inspect the mapping.

11. EXTERNAL VALIDITY AND POSITIONING

This system is not proposed as a new telemetry standard. It is positioned as a transformation layer between OpenTelemetry observability systems and local review-oriented execution evidence artifacts. Within the repository boundary, it turns an OTLP JSON trace into a local operation-accountability statement that can be checked by schema, reference, consistency, and integrity validation.

The system belongs to a class of:

- trace-to-evidence mapping systems;
- observability-to-audit conversion layers;
- evidence packaging tools.

We compare it against:

- raw OTLP trace retention;
- OpenTelemetry trace plus JSON export only;
- log-only observability systems;
- trace visualization systems.

The baseline comparison in 'experiments/results/otel_native_vs_eeoap.md' shows the intended difference. Raw OTLP retention preserves the trace payload but does not bind policy, provenance, evidence references, or validator output. OpenTelemetry JSON export improves portability but remains observability-oriented. The OpenTelemetry-to-EEOAP conversion adds a local operation statement, evidence references, integrity digests, provenance links, and validator-readable output.

The public-facing EEOAP artifact 'docs/public/EEOAP_v0.1_public_spec.md' also narrows the ontology claim: EEOAP v0.1 is a research abstraction and local profile-aware validation surface, not an official standard, legal non-repudiation layer, production deployment proof, or external certification.

12. RELATED WORK POSITIONING

This work is positioned relative to:

1. OpenTelemetry, which provides distributed tracing data rather than schema-bound execution evidence.
2. Jaeger and Zipkin, which provide trace storage and visualization rather than trace-to-evidence transformation.
3. Logging frameworks, which often produce unstructured operational records.
4. Observability platforms, which aggregate telemetry rather than producing a local artifact-level mapping into validator-readable evidence.

EEOAP differs by introducing a deterministic mapping from trace structure to

validated execution evidence within a repository-local validation boundary.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE MANUSCRIPT PREPARATION PROCESS

During the preparation of this work, the author used ChatGPT, Codex, and AI-assisted research tools to support manuscript organization, language refinement, reviewer-risk analysis, and revision planning. After using these tools, the author reviewed, edited, verified, and adapted the content as needed and takes full responsibility for the content of the submitted manuscript.

13. REFERENCES

1. OpenTelemetry. Trace API specification.
<https://opentelemetry.io/docs/specs/otel/trace/api/>
2. OpenTelemetry. Common specification concepts.
<https://opentelemetry.io/docs/specs/otel/common/>
3. open-telemetry/opentelemetry-proto. 'examples/trace.json'.
<https://github.com/open-telemetry/opentelemetry-proto/blob/main/examples/trace.json>
4. open-telemetry/opentelemetry-proto. 'opentelemetry/proto/trace/v1/trace.proto'.
<https://github.com/open-telemetry/opentelemetry-proto/blob/main/opentelemetry/proto/trace/v1/trace.proto>
5. Jaeger Documentation. <https://www.jaegertracing.io/docs/>
6. OpenZipkin. <https://zipkin.io/>